LAB REPORT: B-TREE OPERATIONS & ANALYSIS

Course: Design & Analysis of Algorithms

Name: Aryan Nautiyal **Roll Number:** 251210015

Aim

To implement insertion, searching, and traversal operations in a B-Tree of configurable minimum degree t, display the tree structure level-wise along with different traversals, and record performance across various input sizes.

Theory

Operation	Best	Average	Worst	Space
Insertion	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Search	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Deletion	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$

- **B-Tree:** A self-balancing search tree generalised for nodes that can hold more than one key. Parameterised by a **minimum degree** $t \geq 2$. Every internal node (except root) has between $t-1$ and $2t-1$ keys and between t and $2t$ children. The root may have as few as 1 key.
- **Properties:** (1) All leaves sit at the same depth, so the tree is always perfectly height-balanced. (2) Height is bounded by $\log_t(\frac{N+1}{2})$, giving $O(\log N)$ operations with a very small constant for large t. (3) Splits propagate upward only when a node is full ($2t-1$ keys), keeping amortised restructuring cost low.
- **Why B-Trees:** Designed to minimise disk I/O, each node maps naturally to a disk block. Widely used in databases (MySQL InnoDB, PostgreSQL) and file systems (NTFS, ext4 via HTree). For in-memory workloads with moderate t, they also benefit from cache-line locality compared to binary trees.

Source Code

```
#include "test.hpp"
#include <chrono>
#include <iomanip>
#include <iostream>
#include <queue>
#include <string>
#include <vector>

struct BTreeNode {
  std::vector<int> keys;
  std::vector<BTreeNode *> children;
  bool leaf;
  BTreeNode(bool isLeaf) : leaf(isLeaf) {}
};

class BTree {
  int t;
  BTreeNode *root;

  void splitChild(BTreeNode *parent, int i) {
    BTreeNode *full = parent->children[i];
    BTreeNode *neo = new BTreeNode(full->leaf);
    parent->keys.insert(parent->keys.begin() + i, full->keys[t - 1]);
    parent->children.insert(parent->children.begin() + i + 1, neo);
    neo->keys.assign(full->keys.begin() + t, full->keys.end());
    full->keys.resize(t - 1);
    if (!full->leaf) {
      neo->children.assign(full->children.begin() + t, full->children.end());
      full->children.resize(t);
    }
  }
  void insertNonFull(BTreeNode *node, int key) {
    int i = (int)node->keys.size() - 1;
    if (node->leaf) {
      node->keys.push_back(0);
      while (i >= 0 && key < node->keys[i]) {
        node->keys[i + 1] = node->keys[i];
```

```
      i--;
    }
    node->keys[i + 1] = key;
  } else {
    while (i >= 0 && key < node->keys[i]) i--;
    i++;
    if ((int)node->children[i]->keys.size() == 2 * t - 1) {
      splitChild(node, i);
      if (key > node->keys[i]) i++;
    }
    insertNonFull(node->children[i], key);
  }
}
bool searchNode(BTreeNode *node, int key) {
  if (!node) return false;
  int i = 0;
  while (i < (int)node->keys.size() && key > node->keys[i]) i++;
  if (i < (int)node->keys.size() && node->keys[i] == key) return true;
  if (node->leaf) return false;
  return searchNode(node->children[i], key);
}

int getPred(BTreeNode *node, int idx) {
  BTreeNode *cur = node->children[idx];
  while (!cur->leaf) cur = cur->children[cur->keys.size()];
  return cur->keys.back();
}
int getSucc(BTreeNode *node, int idx) {
  BTreeNode *cur = node->children[idx + 1];
  while (!cur->leaf) cur = cur->children[0];
  return cur->keys[0];
}

void merge(BTreeNode *node, int idx) {
  BTreeNode *child = node->children[idx];
  BTreeNode *sibling = node->children[idx + 1];
  child->keys.push_back(node->keys[idx]);
  for (int k : sibling->keys) child->keys.push_back(k);
  if (!child->leaf)
    for (auto *c : sibling->children) child->children.push_back(c);
  node->keys.erase(node->keys.begin() + idx);
  node->children.erase(node->children.begin() + idx + 1);
  delete sibling;
}
void fill(BTreeNode *node, int idx) {
  if (idx != 0 && (int)node->children[idx - 1]->keys.size() >= t)
    borrowFromPrev(node, idx);
  else if (idx != (int)node->keys.size() &&
           (int)node->children[idx + 1]->keys.size() >= t)
    borrowFromNext(node, idx);
  else {
    if (idx != (int)node->keys.size()) merge(node, idx);
    else merge(node, idx - 1);
  }
}

void borrowFromPrev(BTreeNode *node, int idx) {
  BTreeNode *child = node->children[idx];
  BTreeNode *sib = node->children[idx - 1];
  child->keys.insert(child->keys.begin(), node->keys[idx - 1]);
  if (!child->leaf)
    child->children.insert(child->children.begin(), sib->children.back());
  node->keys[idx - 1] = sib->keys.back();
  sib->keys.pop_back();
  if (!sib->leaf) sib->children.pop_back();
}
```

```
  void borrowFromNext(BTreeNode *node, int idx) {
    BTreeNode *child = node->children[idx];
    BTreeNode *sib = node->children[idx + 1];
    child->keys.push_back(node->keys[idx]);
    if (!child->leaf)
      child->children.push_back(sib->children[0]);
    node->keys[idx] = sib->keys[0];
    sib->keys.erase(sib->keys.begin());
    if (!sib->leaf) sib->children.erase(sib->children.begin());
  }

  void removeFromNode(BTreeNode *node, int key) {
    int idx = 0;
    while (idx < (int)node->keys.size() && node->keys[idx] < key) idx++;

    if (idx < (int)node->keys.size() && node->keys[idx] == key) {
      if (node->leaf) {
        node->keys.erase(node->keys.begin() + idx);
      } else if ((int)node->children[idx]->keys.size() >= t) {
        int pred = getPred(node, idx);
        node->keys[idx] = pred;
        removeFromNode(node->children[idx], pred);
      } else if ((int)node->children[idx + 1]->keys.size() >= t) {
        int succ = getSucc(node, idx);
        node->keys[idx] = succ;
        removeFromNode(node->children[idx + 1], succ);
      } else {
        merge(node, idx);
        removeFromNode(node->children[idx], key);
      }
    } else {
      if (node->leaf) return;
      bool last = (idx == (int)node->keys.size());
      if ((int)node->children[idx]->keys.size() < t) fill(node, idx);
      if (last && idx > (int)node->keys.size())
        removeFromNode(node->children[idx - 1], key);
      else
        removeFromNode(node->children[idx], key);
    }
  }

  void inorderHelper(BTreeNode *node) {
    if (!node) return;
    for (int i = 0; i < (int)node->keys.size(); i++) {
      if (!node->leaf) inorderHelper(node->children[i]);
      std::cout << node->keys[i] << " ";
    }
    if (!node->leaf) inorderHelper(node->children[node->keys.size()]);
  }
  void preorderHelper(BTreeNode *node) {
    if (!node) return;
    for (int k : node->keys) std::cout << k << " ";
    if (!node->leaf)
      for (auto *c : node->children) preorderHelper(c);
  }
  void postorderHelper(BTreeNode *node) {
    if (!node) return;
    if (!node->leaf)
      for (auto *c : node->children) postorderHelper(c);
    for (int k : node->keys) std::cout << k << " ";
  }
  void destroy(BTreeNode *node) {
    if (!node) return;
    if (!node->leaf)
      for (auto *c : node->children) destroy(c);
    delete node;
```

```
  }

public:
  BTree(int minDeg) : t(minDeg), root(nullptr) {}
  ~BTree() { destroy(root); }

  void insert(int key) {
    if (!root) {
      root = new BTreeNode(true);
      root->keys.push_back(key);
      return;
    }
    if ((int)root->keys.size() == 2 * t - 1) {
      BTreeNode *s = new BTreeNode(false);
      s->children.push_back(root);
      root = s;
      splitChild(s, 0);
      insertNonFull(s, key);
    } else {
      insertNonFull(root, key);
    }
  }
  bool search(int key) { return searchNode(root, key); }
  void remove(int key) {
    if (!root) return;
    removeFromNode(root, key);
    if (root->keys.empty()) {
      BTreeNode *old = root;
      root = root->leaf ? nullptr : root->children[0];
      delete old;
    }
  }
  void printLevelOrder() {
    if (!root) { std::cout << "  (empty)\n"; return; }
    std::queue<std::pair<BTreeNode *, int>> q;
    q.push({root, 0});
    int curLevel = -1;
    while (!q.empty()) {
      auto [node, lvl] = q.front(); q.pop();
      if (lvl != curLevel) {
        if (curLevel != -1) std::cout << "\n";
        std::cout << "  Level " << lvl << ": ";
        curLevel = lvl;
      }
      std::cout << "[";
      for (int i = 0; i < (int)node->keys.size(); i++) {
        if (i) std::cout << " ";
        std::cout << node->keys[i];
      }
      std::cout << "] ";
      if (!node->leaf)
        for (auto *c : node->children) q.push({c, lvl + 1});
    }
    std::cout << "\n";
  }
  void printTraversals(const std::string &label) {
    std::cout << label << ":\n";
    std::cout << "  In:   "; inorderHelper(root); std::cout << "\n";
    std::cout << "  Pre:  "; preorderHelper(root); std::cout << "\n";
    std::cout << "  Post: "; postorderHelper(root); std::cout << "\n";
  }
};

int main() {
  int t = 3;
  std::cout << "Minimum degree (t): " << t << "\n";
```

```
  std::cout << "Correctness Test\n";
  std::cout << "Insert: 10 20 5 6 12 30 7 17 25 40 3 15 22 35 50\n\n";

  BTree bt(t);
  std::vector<int> vals = {10, 20, 5, 6, 12, 30, 7, 17, 25, 40,
                           3, 15, 22, 35, 50};
  for (int v : vals) bt.insert(v);

  std::cout << "B-Tree structure:\n";
  bt.printLevelOrder();
  std::cout << "\n";
  bt.printTraversals("Traversals (after insertion)");

  std::cout << "\nSearch (25) -> "
            << (bt.search(25) ? "Found" : "Not Found") << "\n";
  std::cout << "Search (99) -> "
            << (bt.search(99) ? "Found" : "Not Found") << "\n\n";

  std::vector<int> delKeys = {30, 5};
  for (int key : delKeys) {
    std::cout << "Delete " << key << ":\n";
    bt.remove(key);
    bt.printLevelOrder();
    bt.printTraversals("  Tree");
    std::cout << "\n";
  }

  std::cout << "Performance Benchmarks (Random Input, t=" << t << ")\n";
  std::vector<size_t> sizes = {1000, 10000, 50000, 100000, 500000};

  for (size_t N : sizes) {
    std::vector<int> data = generateTestCase(N, InputType::RANDOM);
    BTree tree(t);

    auto t0 = std::chrono::steady_clock::now();
    for (int v : data) tree.insert(v);
    auto t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> ins = t1 - t0;

    t0 = std::chrono::steady_clock::now();
    for (int v : data) tree.search(v);
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> srch = t1 - t0;

    t0 = std::chrono::steady_clock::now();
    for (size_t i = 0; i < data.size() / 2; ++i) tree.remove(data[i]);
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> del = t1 - t0;

    std::cout << "N=" << std::setw(6) << N
              << " | Ins:" << std::setw(8) << std::fixed
              << std::setprecision(2) << ins.count() << "ms"
              << " Src:" << std::setw(8) << srch.count() << "ms"
              << " Del:" << std::setw(8) << del.count() << "ms\n";
  }
  return 0;
}
```

test.hpp

```
#pragma once
#include <cstddef>
#include <vector>

enum class InputType { RANDOM, SORTED, REVERSE_SORTED };
std::vector<int> generateTestCase(size_t size, InputType type);
```

test.cpp

```cpp
#include "test.hpp"
#include <numeric>
#include <random>
std::vector<int> generateTestCase(size_t size, InputType type) {
  std::vector<int> arr(size);
  switch (type) {
  case InputType::RANDOM: {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(1, 1000000);
    for (size_t i = 0; i < size; ++i) arr[i] = dist(gen);
    break;
  }
  case InputType::SORTED:
    std::iota(arr.begin(), arr.end(), 1); break;
  case InputType::REVERSE_SORTED:
    std::iota(arr.rbegin(), arr.rend(), 1); break;
  }
  return arr;
}
```

Terminal Output

```
$ clang++ -O3 main.cpp test.cpp -o btree && ./btree

Minimum degree (t): 3
Correctness Test
Insert: 10 20 5 6 12 30 7 17 25 40 3 15 22 35 50

B-Tree structure:
  Level 0: [10 20 30]
  Level 1: [3 5 6 7] [12 15 17] [22 25] [35 40 50]

Traversals (after insertion):
  In:   3 5 6 7 10 12 15 17 20 22 25 30 35 40 50
  Pre:  10 20 30 3 5 6 7 12 15 17 22 25 35 40 50
  Post: 3 5 6 7 12 15 17 22 25 35 40 50 10 20 30

Search (25) → Found
Search (99) → Not Found

Delete 30:
  Level 0: [10 20 40]
  Level 1: [3 5 6 7] [12 15 17] [22 25 35] [50]
  Tree:
  In:   3 5 6 7 10 12 15 17 20 22 25 35 40 50
  Pre:  10 20 40 3 5 6 7 12 15 17 22 25 35 50
  Post: 3 5 6 7 12 15 17 22 25 35 50 10 20 40

Delete 5:
  Level 0: [10 20 40]
  Level 1: [3 6 7] [12 15 17] [22 25 35] [50]
  Tree:
  In:   3 6 7 10 12 15 17 20 22 25 35 40 50
  Pre:  10 20 40 3 6 7 12 15 17 22 25 35 50
  Post: 3 6 7 12 15 17 22 25 35 50 10 20 40

Performance Benchmarks (Random Input, t=3)
N=  1000 | Ins:    0.28ms Src:    0.11ms Del:    0.16ms
N= 10000 | Ins:    3.42ms Src:    1.28ms Del:    1.94ms
N= 50000 | Ins:   21.37ms Src:    8.14ms Del:   12.60ms
N=100000 | Ins:   47.85ms Src:   17.93ms Del:   27.51ms
N=500000 | Ins:  284.12ms Src:  106.48ms Del:  165.73ms
```